

Python Code

```
1  # -*- coding: utf-8 -*-
2  """
3
4
5  Assignment submitted by:
6  Chandrika Mysore Rameshbabu
7
8  Student ID:
9  2026AJ05217
10
11
12  **MFML Assignment 1 **
13
14  Q1) Finding solutions of linear systems
15  1. Write a code taking as input a matrix A of size mxn and a vector b of size mx1, where
16  m and n are arbitrarily large numbers and  $m < n$ , constructing the augmented matrix
17  and performing
18  REF, and
19  RREF
20  without using any built-in functions. In case you encounter any division by 0, you can
21  choose a different A and/or b. Note that this part should be explicitly there in the
22  code
23  and give a comment line on the same.
24  2. Write a Python code to identify the pivot and non-pivot columns and find the
25  particular
26  solution and solutions to  $Ax = 0$ .
27  3. Consider a random 5 x 7 matrix A and a suitable b and show the REF, RREF, pivot
28  columns, non-pivot columns, the particular solution, the solutions to  $Ax = 0$ , the general
29  solution and verify the general solution.
30
31  Complete solution:
32  """
33
34  #-----
35  # Question 1 a - Construct augmented matrix, compute REF and RREF
36  #-----
37
38  # Prerequisite and miscellaneous methods
39
40  import numpy as np
41
42  # Since we are considering float values, we will consider a very small value
43  # to check for zero pivot elements
44  almost_zero = 1e-10
45
46  # This is to avoid division by 0, and when we encounter a zero pivot,
47  # we will swap the row with a non-zero pivot row below it
48  # We can choose a different matrix, but we will do so if we encounter a zero
49  # pivot in the last column, since that means the system is inconsistent and has
50  # no solution
51  def check_non_zero_pivot_row(mat, row, colIndex):
52      for r in range(row, mat.shape[0]):
53          # check for the elements of each of the next row for the given column index
54          if abs(mat[r,colIndex]) > almost_zero:
55              return r # if pivot is greater than zero, return the row index
```

```

56     return None # if no pivot element is found, return None
57
58
59 # Swap the rows row1 and row2 of the matrix
60 def swap_rows(matrix, row1, row2):
61     if(row1 != row2):
62         matrix[[row1, row2], :] = matrix[[row2, row1], :]
63
64 # construct the augmented matrix from A and b
65 def get_augmented_matrix(A, b):
66     return np.hstack(
67         [np.asarray(A, dtype=float), np.asarray(b, dtype=float).reshape(-1, 1)]
68     )
69
70 # Check for m < n
71 def check_size(A):
72     m,n = A.shape
73     # Enforcing m < n
74     if(m >= n):
75         print("\nNumber of rows should be less than number of columns.")
76         print("\nExiting the program")
77         exit() # We are considering m < n, hence exiting if not satisfied
78
79 # Compute REF of the augmented matrix [A|b]
80 def do_ref(A,b):
81     check_size(A) # Ensuring rows is less than columns
82
83     aug = get_augmented_matrix(A, b) # Use the method implmented above
84     print("\nAugmented matrix is")
85     print(aug)
86
87     rows, cols = aug.shape
88     pivot_index = 0
89
90     for col in range(cols-1):
91         if pivot_index >= rows:
92             break # Stop execution if we have already processed all the rows
93         target_row = check_non_zero_pivot_row(aug, pivot_index, col)
94         if target_row is None:
95             continue
96
97         swap_rows(aug, pivot_index, target_row)
98         pivot_value = aug[pivot_index, col]
99         #If pivot value is zero, we cannot divide by zero,
100         #hence we will skip this column and move to the next column
101         if(abs(pivot_value) < almost_zero):
102             continue
103
104         # Convert pivot element to 1 by dividing the entire row by the pivot value
105         aug[pivot_index, col:] = aug[pivot_index, col:] / pivot_value
106
107         for r in range(pivot_index+1, rows):
108             # Considering the factor to eliminate the elements below the pivot element
109             factor = aug[r, col]
110             if(abs(factor) > almost_zero):
111                 # column elimination to make the elements below the pivot element zero
112                 aug[r, col:] = aug[r, col:] - factor*aug[pivot_index, col:]
113
114         pivot_index = pivot_index + 1 # Move to enxt row

```

```

115
116     return aug
117
118 # Compute RREF of the augmented matrix [A|b]
119 def do_rref(A,b):
120     check_size(A) # Ensuring rows is less than columns
121
122     aug = get_augmented_matrix(A, b)
123     rows, cols = aug.shape
124
125     pivot_index = 0
126
127     for col in range(cols-1):
128         if(pivot_index >= rows):
129             break # Stop execution if we have already processed all the rows
130
131         # Check for a non-zero pivot element in the current column,
132         # starting from the current pivot_index row
133         target_row = check_non_zero_pivot_row(aug, pivot_index, col)
134         if(target_row is None):
135             continue # If no non-zero pivot element is found, move to the next column
136
137         # Swap current row with target row to bring non-zero pivot to current row
138         swap_rows(aug, pivot_index, target_row)
139         pivot_value=aug[pivot_index, col]
140         if(abs(pivot_value) < almost_zero):
141             # If pivot value is zero, we cannot divide by zero,
142             # hence we will skip this column and move to the next column
143             continue
144
145         #Convert pivot element to 1 by dividing the entire row by the pivot value
146         aug[pivot_index, col:] = aug[pivot_index, col:] / pivot_value
147
148         for r in range(rows):
149             # If current row is pivot row, we will skip it and move to the next row
150             if r == pivot_index:
151                 continue # Skip the pivot row
152
153             # Considring factor to eliminate elements above and below pivot element
154             factor = aug[r,col]
155             if(abs(factor) > almost_zero):
156                 # column elimination to make above and below pivot elements to zero
157                 aug[r, col:] = aug[r, col:] - factor*aug[pivot_index, col:]
158
159         pivot_index += 1
160
161     return aug
162
163
164
165 # Checking output of 1a using a random matrix and vector, where m < n
166 print("Enter the size of the matrix A = m x n")
167 m = int(input("m = "))
168 n = int(input("n = "))
169
170 while m >= n:
171     print("The condition m < n is required. Please re-enter valid sizes.")
172     m = int(input("m = "))
173     n = int(input("n = "))

```

```

174
175
176 rng = np.random.default_rng(25)
177 A = np.array([[rng.uniform(0.1, 9.9) for _ in range(n)] for _ in range(m)], float)
178 b = np.array([rng.uniform(0.1, 9.9) for _ in range(m)], float)
179
180 print("\n\n")
181 print("=" * 70)
182 print("Demonstrating solution to Question 1a")
183 print("=" * 70)
184 print("\nConsidering a random matrix A of size ", m, " * ", n, " and a random vector b
of size ", m, " * ", "1,\n\n")
185 print("Random matrix generated for A with ", m, " * ", n, "is:")
186 print(A)
187 print("\nRandom vector b generated with size ", m, " * ", "1 is:")
188 print(b)
189
190 print("\n")
191 ref = do_ref(A,b)
192 rref = do_rref(A,b)
193
194 print("\nREF of Matrix [A|b] is:")
195 print(ref)
196 print("\nRREF of Matrix [A|b] is:")
197 print(rref)
198
199
200
201 ""
202 Output of 1a:
203
204 Enter the size of the matrix A = m x n
205 m = 4
206 n = 2
207 The condition m < n is required. Please re-enter valid sizes.
208 m = 3
209 n = 5
210
211 =====
212 Demonstrating solution to Question 1a
213 =====
214
215 Considering a random matrix A of size 3 * 5 and a random vector b of size 3 * 1,
216
217 Random matrix generated for A with 3 * 5 is:
218 [[1.67506809 0.10305794 2.22266495 3.70668711 0.11985684]
219  [1.99221223 9.81044935 7.81195596 1.28855505 2.31255889]
220  [7.61490021 9.08501469 3.5674528 7.79993094 1.53784883]]
221
222 Random vector b generated with size 3 * 1 is:
223 [5.40893986 5.94923938 7.8410283 ]
224
225 Augmented matrix is
226 [[1.67506809 0.10305794 2.22266495 3.70668711 0.11985684 5.40893986]
227  [1.99221223 9.81044935 7.81195596 1.28855505 2.31255889 5.94923938]
228  [7.61490021 9.08501469 3.5674528 7.79993094 1.53784883 7.8410283 ]]
229
230 REF of Matrix [A|b] is:
231 [[ 1.          0.06152463  1.32691021  2.21285758  0.07155341  3.22908657]

```

```

232 [ 0.          1.          0.53349852 -0.32204436  0.22399219 -0.04993728]
233 [ 0.          0.          1.          0.56367983  0.08416355  1.46562346]]
234
235 RREF of Matrix [A|b] is:
236 [[ 1.          0.          0.          1.50322055 -0.05114257  1.3355148 ]
237 [ 0.          1.          0.         -0.62276671  0.17909106 -0.83184523]
238 [ 0.          0.          1.          0.56367983  0.08416355  1.46562346]]
239
240
241 """
242
243
244 #-----
245 ## Question 1 b - Code to identify pivot columns and non pivot columns, particular
246 solution and solutions to  $Ax = 0$  which is null space vectors
247 #-----
248
249 # Identify pivot and free columns from RREF
250 def get_pivot_and_free_columns(rref):
251     pivot_columns = []
252     free_columns = []
253     for row in range(rref.shape[0]):
254         for col in range(rref.shape[1]):
255             if(abs(rref[row,col]) > almost_zero):
256                 pivot_columns.append(col)
257                 break
258     free_columns = [j for j in range(A.shape[1]) if j not in pivot_columns]
259     return pivot_columns, free_columns
260
261 def get_particular_solution(rref, pivot_columns):
262     particular_solution = np.zeros(A.shape[1], float)
263     for row, pivot_col in enumerate(pivot_columns):
264         if(row < rref.shape[0]):
265             particular_solution[pivot_col] = rref[row,-1] # The last column of the RREF matrix
266 is the solution vector, hence we will assign the values of the last column to the
267 corresponding pivot columns in the particular solution vector
268     return particular_solution
269
270 def get_null_space_vectors(rref, pivot_columns, free_columns):
271     null_space_vectors = []
272     for free_col in free_columns:
273         v = np.zeros(A.shape[1], dtype=float) # Initialize a vector for the null space
274         v[free_col] = 1.0
275         for row, pivot_col in enumerate(pivot_columns):
276             v[pivot_col] = -rref[row, free_col] # The negative of the corresponding entry in
277 the RREF matrix for the free column is assigned to the pivot column in the null space
278 vector
279         null_space_vectors.append(v) # Add the null space vector to the list
280     return null_space_vectors
281
282 # Check output of 1b using the RREF matrix obtained from 1a
283 pivot_columns, free_columns = get_pivot_and_free_columns(rref)
284
285 if(pivot_columns[-1] == A.shape[1]): # This means that last column from the augment
286 matrix is pivot column, which means system is inconsistent and has no solution, and exit
287 program
288     print("\nThe system is inconsistent since there is a pivot in the last column, hence
289 no solution exists. Choose another matrix.\n")

```

```

283     exit()
284
285 print("\n\n")
286 print("="*70)
287 print("Demonstrating solution to Question 1b - Identify the pivot and non-pivot columns
and find the particular solution and solutions to  $Ax = 0$ :")
288 print("="*70)
289 print("\nPivot columns are ", pivot_columns)
290 print("Non pivot columns are ", free_columns)
291
292 particular_solution = get_particular_solution(rref, pivot_columns)
293 print("\nParticular solution is ", particular_solution)
294
295 null_space_vectors = get_null_space_vectors(rref, pivot_columns, free_columns)
296 print("\nSolutions to  $Ax = 0$ :")
297 for i, vec in enumerate(null_space_vectors): # Print each null space vector
298     print(f"v{i+1} =", vec)
299
300
301 """
302 Output of 1 b
303
304 =====
305 Demonstrating solution to Question 1b - Identify the pivot and non-pivot columns and
find the particular solution and solutions to  $Ax = 0$ :
306 =====
307
308 Pivot columns are [0, 1, 2]
309 Non pivot columns are [3, 4]
310
311 Particular solution is [ 1.3355148 -0.83184523 1.46562346 0.          0.          ]
312
313 Solutions to  $Ax = 0$ :
314 v1 = [-1.50322055  0.62276671 -0.56367983  1.          0.          ]
315 v2 = [ 0.05114257 -0.17909106 -0.08416355  0.          1.          ]
316
317
318 """
319
320
321 #-----
322 ## Question 1 c - Demonstrate the solution to question 1a and 1b,
323 ## by considering a random 5*7 matrix A and a random 5*1 vector b, where  $m < n$ 
324 #-----
325
326 # Taking m and n as 5 and 7 as per question
327 m = 5
328 n = 7
329
330 # Create the random matrix of m*n and the vector m*1
331 rng = np.random.default_rng(25)
332 A = np.array([[rng.uniform(0.1, 9.9) for _ in range(n)] for _ in range(m)], float)
333 b = np.array([rng.uniform(0.1, 9.9) for _ in range(m)], float)
334
335 print("\n\n")
336 print("=" * 70)
337 print("Demonstrating solution to Question 1c, that uses the code written in 1a and 1b
above:")
338 print("=" * 70)

```

```

339 print("\nConsidering a random 5*7 matrix A and a random 5*1 vector b,")
340 print("to demonstrate the solution to question 1a and 1b, where m < n\n")
341 print("Random matrix generated for A with ", m, " * ", n, "is:")
342 print(A)
343 print("\nRandom vector b generated with size ", m, " * ", "1 is:")
344 print(b)
345
346 print("\n")
347 print("=" * 70)
348 print("Demonstrating solution to Question 1a - REF and RREF of the augmented matrix [A|
b] are:")
349 print("=" * 70)
350 ref = do_ref(A,b)
351 rref = do_rref(A,b)
352
353 print("\nREF of Matrix [A|b] is:")
354 print(ref)
355 print("\nRREF of Matrix [A|b] is:")
356 print(rref)
357
358 pivot_columns, free_columns = get_pivot_and_free_columns(rref)
359
360 if(pivot_columns[-1] == A.shape[1]): # This means that last column from the augment
matrix is pivot column, which means system is inconsistent and has no solution, and exit
program
361     print("\nThe system is inconsistent since there is a pivot in the last column, hence
no solution exists. Choose another matrix..\n")
362     exit()
363
364 print("\n\n")
365 print("="*70)
366 print("Demonstrating solution to Question 1b - Identify the pivot and non-pivot columns
and find the particular solution and solutions to Ax= 0:")
367 print("="*70)
368 print("\nPivot columns are ", pivot_columns)
369 print("Non pivot columns are ", free_columns)
370
371 particular_solution = get_particular_solution(rref, pivot_columns)
372 print("\nParticular solution is ", particular_solution)
373
374 null_space_vectors = get_null_space_vectors(rref, pivot_columns, free_columns)
375 print("\nSolutions to A x = 0:")
376 for i, vec in enumerate(null_space_vectors): # Print each null space vector
377     print(f"v{i+1} =", vec)
378
379 # Finally verify the general solution.
380 # General solution = x_particular + sum(c_i * v_i)
381 print("\nGeneral solution of form x = particular_solution + c1*v1 + c2*v2 + ...:")
382 print(particular_solution, " + \n", " + \n".join([f"c{i+1}* {null_space_vectors[i]}"
for i in range(len(null_space_vectors))]))
383
384 # Verify the general solution by substituting random variables to c
385 random_coefficients = np.random.uniform(-10, 10, len(null_space_vectors))
386
387 # Generate random coefficients for the null space vectors
388 general_solution = particular_solution.copy()
389
390 for i, coeff in enumerate(random_coefficients):
391     general_solution += coeff * null_space_vectors[i]

```

```

392     # compute the general solution by adding the particular solution and the linear
    combination of null space vectors with random coefficients
393
394
395 def matmul(A, B):
396     A = np.asarray(A, dtype=float)
397     B = np.asarray(B, dtype=float)
398
399     # Case 1: vector . vector -> scalar (dot product)
400     if A.ndim == 1 and B.ndim == 1:
401         if A.shape[0] != B.shape[0]:
402             raise ValueError("Vectors must be of the same length for dot product.")
403         total = 0.0
404         for i in range(A.shape[0]):
405             total += A[i] * B[i]
406         return total
407
408     # Case 2: matrix . vector -> vector
409     if A.ndim == 2 and B.ndim == 1:
410         rows, cols = A.shape
411         if cols != B.shape[0]:
412             raise ValueError("Matrix columns must match vector length.")
413         result = np.zeros(rows, dtype=float)
414         for i in range(rows):
415             s = 0.0
416             for k in range(cols):
417                 s += A[i, k] * B[k]
418             result[i] = s
419         return result
420
421     # Case 3: vector . matrix -> vector
422     if A.ndim == 1 and B.ndim == 2:
423         length = A.shape[0]
424         rows, cols = B.shape
425         if length != rows:
426             raise ValueError("Vector length must match matrix rows.")
427         result = np.zeros(cols, dtype=float)
428         for j in range(cols):
429             s = 0.0
430             for k in range(length):
431                 s += A[k] * B[k, j]
432             result[j] = s
433         return result
434
435     # Case 4: matrix . matrix -> matrix
436     if A.ndim == 2 and B.ndim == 2:
437         rowsA, colsA = A.shape
438         rowsB, colsB = B.shape
439         if colsA != rowsB:
440             raise ValueError("Inner dimensions must match for matrix multiplication.")
441         result = np.zeros((rowsA, colsB), dtype=float)
442         for i in range(rowsA):
443             for j in range(colsB):
444                 s = 0.0
445                 for k in range(colsA):
446                     s += A[i, k] * B[k, j]
447                 result[i, j] = s
448         return result
449

```

```

450         raise ValueError("Unsupported input dimensions for matmul.")
451
452
453     print("\n\nVerifying general solution by substituting random coefficients to c:")
454     print("\nChosen random coefficients for null space vectors:\n", random_coefficients)
455     print("One of the general solution with the above random coefficient:\n",
456           general_solution)
457     print("Verifying general solution - calculated target vectors using the above general
458           solution=\n", matmul(A, general_solution))
459     print("Which is equal to b =\n", b, "\nHence general solution is verified.", "\n\n")
460
461     """
462     Output:
463
464     =====
465     Demonstrating solution to Question 1c, that uses the code written in 1a and 1b above:
466     =====
467
468     Considering a random 5*7 matrix A and a random 5*1 vector b,
469     to demonstrate the solution to question 1a and 1b, where m < n
470
471     Random matrix generated for A with 5 * 7 is:
472     [[1.67506809 0.10305794 2.22266495 3.70668711 0.11985684 1.99221223
473        9.81044935]
474      [7.81195596 1.28855505 2.31255889 7.61490021 9.08501469 3.5674528
475        7.79993094]
476      [1.53784883 5.40893986 5.94923938 7.8410283 2.8305963 4.15360841
477        3.78549439]
478      [1.45150246 6.72430849 5.56212287 5.06057701 6.23883848 6.03153557
479        0.81693044]
480      [3.03761958 4.05271324 1.94553782 7.16358112 6.70710414 7.61624006
481        0.25840532]]
482
483     Random vector b generated with size 5 * 1 is:
484     [1.0547383 2.49134204 5.40660293 0.8594453 9.1875643 ]
485
486
487     =====
488     Demonstrating solution to Question 1a - REF and RREF of the augmented matrix [A|b] are:
489     =====
490
491     Augmented matrix is
492     [[1.67506809 0.10305794 2.22266495 3.70668711 0.11985684 1.99221223
493        9.81044935 1.0547383 ]
494      [7.81195596 1.28855505 2.31255889 7.61490021 9.08501469 3.5674528
495        7.79993094 2.49134204]
496      [1.53784883 5.40893986 5.94923938 7.8410283 2.8305963 4.15360841
497        3.78549439 5.40660293]
498      [1.45150246 6.72430849 5.56212287 5.06057701 6.23883848 6.03153557
499        0.81693044 0.8594453 ]
500      [3.03761958 4.05271324 1.94553782 7.16358112 6.70710414 7.61624006
501        0.25840532 9.1875643 ]]
502
503     REF of Matrix [A|b] is:
504     [[ 1.          0.06152463  1.32691021  2.21285758  0.07155341
505        1.18933209  5.85674662  0.62966891]
506      [ 0.          1.          -9.96773448 -11.97118257 10.5529818

```

```

507     -7.08424729 -46.97540682 -3.00473029]
508 [ 0.          0.          1.          1.19648778 -0.93813309
509     0.70274761  4.29710655  0.35875924]
510 [ 0.          0.          0.          1.          -0.7129787
511     -1.03316715 -1.89651009  2.33646124]
512 [ 0.          0.          0.          0.          1.
513     4.27099896  6.33924061 -0.69006639]]
514
515 RREF of Matrix [A|b] is:
516 [[ 1.          0.          0.          0.          0.
517     -6.30814599 -9.11361855 -0.18246482]
518 [ 0.          1.          0.          0.          0.
519     -5.12248835 -11.64444166  1.48352608]
520 [ 0.          0.          1.          0.          0.
521     2.30222256  7.10549117 -2.49548705]
522 [ 0.          0.          0.          1.          0.
523     2.01196414  2.62323345  1.8444586 ]
524 [ 0.          0.          0.          0.          1.
525     4.27099896  6.33924061 -0.69006639]]
526
527
528
529 =====
530 Demonstrating solution to Question 1b - Identify the pivot and non-pivot columns and
531 find the particular solution and solutions to Ax= 0:
532 =====
533 Pivot columns are [0, 1, 2, 3, 4]
534 Non pivot columns are [5, 6]
535
536 Particular solution is [-0.18246482  1.48352608 -2.49548705  1.8444586 -0.69006639  0.
537     0.          ]
538
539 Solutions to A x = 0:
540 v1 = [ 6.30814599  5.12248835 -2.30222256 -2.01196414 -4.27099896  1.
541     0.          ]
542 v2 = [ 9.11361855 11.64444166 -7.10549117 -2.62323345 -6.33924061  0.
543     1.          ]
544
545 General solution of form x = particular_solution + c1*v1 + c2*v2 + ...:
546 [-0.18246482  1.48352608 -2.49548705  1.8444586 -0.69006639  0.
547     0.          ] +
548 c1* [ 6.30814599  5.12248835 -2.30222256 -2.01196414 -4.27099896  1.
549     0.          ] +
550 c2* [ 9.11361855 11.64444166 -7.10549117 -2.62323345 -6.33924061  0.
551     1.          ]
552
553
554 Verifying general solution by substituting random coefficients to c:
555
556 Chosen random coefficients for null space vectors:
557 [-6.52782622 -3.66784159]
558 One of the general solution with the above random coefficient:
559 [-74.78825477 -74.66515513  38.59483778  24.59981564  50.44160299
560     -6.52782622 -3.66784159]
561 Verifying general solution - calculated target vectors using the above general solution=
562 [1.0547383  2.49134204  5.40660293  0.8594453  9.1875643 ]
563 Which is equal to b =
564 [1.0547383  2.49134204  5.40660293  0.8594453  9.1875643 ]

```

```

565 Hence general solution is verified.
566
567 """
568
569 """
570 Q2) Consider a dataset  $X \in \mathbb{R}^{500 \times 6}$  constructed as follows: the first four features
571  $f_1, f_2, f_3, f_4$ 
572 are generated as random features sampled from a standard normal distribution (read about
573 this). The fifth and sixth features are defined by the relations
574  $f_5 = 2f_1 + 3f_2$ ,  $f_6 = f_3 - 2f_4$ .
575 Perform the following tasks in sequence:
576 1. Write a Python code to generate the dataset  $X = [f_1, f_2, f_3, f_4, f_5, f_6]$ . [0.5]
577 2. Write a Python code which computes the rank of  $X$  and display the output for the
578 dataset
579  $X$  generated in step 1. [0.5]
580 3. Numerical Experiment with the Power Method
581 Read about the power method for finding the dominant eigenvalue and its corresponding
582 eigenvector and perform the following tasks.
583 (a) Write a Python code to compute the covariance matrix [0.5]
584 1
585  $C =$ 
586  $X^T X.$ 
587  $n$ 
588 where  $n$  is number of data points
589 (b) Implement the Power Method in Python to approximate the largest eigenvalue  $\lambda_1$ 
590 and its corresponding eigenvector  $v_1$  of  $C$ . Show the code and the outputs. [1]
591 (c) Write a Python code to obtain the next largest eigenvalue  $\lambda_2$  and its correspond-
592 ing eigenvector  $v_2$  by applying power method on  $C - v_1 v_1^T$ 
593 1  $C$ . Having found out
594  $v_1, v_2, \dots, v_{k-1}$ , one can find  $\lambda_k$  and its corresponding eigenvector  $v_k$  by applying
595 power method on  $C -$ 
596  $\sum_{j=1}^{k-1} v_j v_j^T$ 
597  $J C$ . Give the code and also display the obtained
598 eigenvalues and the corresponding eigenvectors. [1.5]
599 (d) Find all the eigenvalues and eigenvectors using Python function and compare with
600 the obtained result in (c). [0.5]
601 (e) Compare the number of iterations required to get an accuracy of  $10^{-7}$  using the power
602 method. The actual values can be taken as the one obtained in (d). [0.5]
603
604 Complete solution:
605 """
606
607 # -  $f_1, f_2, f_3, f_4$  means  $N(0,1)$ 
608 # -  $f_5 = 2f_1 + 3f_2$ 
609 # -  $f_6 = f_3 - 2f_4$ 
610 import numpy as np
611
612 np.random.seed(42)
613
614 # -----
615 # 1. Generate the dataset  $X = [f_1, f_2, f_3, f_4, f_5, f_6]$ 
616 # -----
617
618 n = 500
619
620 # Generate the first four original features from the "standard normal distribution".
621 f1 = np.random.randn(n)

```

```

622 f2 = np.random.randn(n)
623 f3 = np.random.randn(n)
624 f4 = np.random.randn(n)
625
626 # Define the dependent features using the given linear relations.
627 f5 = 2.0 * f1 + 3.0 * f2
628 f6 = f3 - 2.0 * f4
629
630 # Stack them together to form the full dataset X with 500 rows and 6 columns.
631 X = np.column_stack([f1, f2, f3, f4, f5, f6])
632
633 print("\n\n")
634 print("=" * 70)
635 print("PART 1: Dataset generation")
636 print("=" * 70)
637 print("Shape of X:", X.shape)
638 print("First 5 rows of X:\n", X[:5])
639
640 print(" \n-> The first four columns are independent random features sampled from a
        standard normal distribution.")
641 print(" -> The last two columns are not independent because they are linear combinations
        of the first four.")
642 print(" -> X is a 500 x 6 matrix with maximum rank 4 according to theory.")
643
644
645
646 # -----
647 # Q2(2): Rank of the dataset X
648 # -----
649
650 almost_zero = 1e-10
651
652 def check_non_zero_pivot_row(mat, row, colIndex):
653     for r in range(row, mat.shape[0]):
654         # check for the elements of each of the next row for the given column index
655         if abs(mat[r,colIndex]) > almost_zero:
656             return r # if pivot is greater than zero, return the row index
657     return None # if no pivot element is found, return None
658
659
660 # Swap the rows row1 and row2 of the matrix
661 def swap_rows(matrix, row1, row2):
662     if(row1 != row2):
663         matrix[[row1, row2], :] = matrix[[row2, row1], :]
664
665
666 # Compute RREF
667 def do_rref(A):
668     aug = A.copy() # Create a copy of the input matrix to avoid modifying it directly
669     rows, cols = aug.shape
670
671     pivot_index = 0
672
673     for col in range(cols-1):
674         if(pivot_index >= rows):
675             break # Stop execution if we have already processed all the rows
676
677         # Check for a non-zero pivot element in the current column,
678         # starting from the current pivot_index row

```

```

679     target_row = check_non_zero_pivot_row(aug, pivot_index, col)
680     if(target_row is None):
681         continue # If no non-zero pivot element is found, move to the next column
682
683     # Swap current row with target row to bring non-zero pivot to current row
684     swap_rows(aug, pivot_index, target_row)
685     pivot_value=aug[pivot_index, col]
686     if(abs(pivot_value) < almost_zero):
687         # If pivot value is zero, we cannot divide by zero,
688         # hence we will skip this column and move to the next column
689         continue
690
691     #Convert pivot element to 1 by dividing the entire row by the pivot value
692     aug[pivot_index, col:] = aug[pivot_index, col:] / pivot_value
693
694     for r in range(rows):
695         # If current row is pivot row, we will skip it and move to the next row
696         if r == pivot_index:
697             continue # Skip the pivot row
698
699         # Considering factor to eliminate elements above and below pivot element
700         factor = aug[r,col]
701         if(abs(factor) > almost_zero):
702             # column elimination to make above and below pivot elements to zero
703             aug[r, col:] = aug[r, col:] - factor*aug[pivot_index, col:]
704
705     pivot_index += 1
706
707     return aug
708
709
710 rrefX = do_rref(X) # Using RREF to verify rank
711
712 pivot_columns= []
713
714 for row in range(rrefX.shape[0]):
715     for col in range(rrefX.shape[1]):
716         if(abs(rrefX[row,col]) > almost_zero):
717             pivot_columns.append(col)
718             break
719
720 rankX = len(pivot_columns)
721
722 print("\n" + "=" * 70)
723 print("PART 2: Rank of X")
724 print("=" * 70)
725 print("Computed Rank from program of X:", rankX)
726 print("(Expected also 4, since f5 and f6 are linear combinations of f1..f4,")
727 print(" so X has only 4 linearly independent columns.)")
728
729
730 # -----
731 # Q2(3a): Compute the covariance matrix C = (1/n) * X^T * X
732 # -----
733 # Covariance measures how features vary together.
734 # For the dataset X of shape (n_rows, 6), the covariance matrix is computed as:
735 # C = (1 / n) * X^T * X
736 # where n = number of data points = 500.
737

```

```

738
739 def matmul(A, B):
740     A = np.asarray(A, dtype=float)
741     B = np.asarray(B, dtype=float)
742
743     # Case 1: vector . vector -> scalar (dot product)
744     if A.ndim == 1 and B.ndim == 1:
745         if A.shape[0] != B.shape[0]:
746             raise ValueError("Vectors must be of the same length for dot product.")
747         total = 0.0
748         for i in range(A.shape[0]):
749             total += A[i] * B[i]
750         return total
751
752     # Case 2: matrix . vector -> vector
753     if A.ndim == 2 and B.ndim == 1:
754         rows, cols = A.shape
755         if cols != B.shape[0]:
756             raise ValueError("Matrix columns must match vector length.")
757         result = np.zeros(rows, dtype=float)
758         for i in range(rows):
759             s = 0.0
760             for k in range(cols):
761                 s += A[i, k] * B[k]
762             result[i] = s
763         return result
764
765     # Case 3: vector . matrix -> vector
766     if A.ndim == 1 and B.ndim == 2:
767         length = A.shape[0]
768         rows, cols = B.shape
769         if length != rows:
770             raise ValueError("Vector length must match matrix rows.")
771         result = np.zeros(cols, dtype=float)
772         for j in range(cols):
773             s = 0.0
774             for k in range(length):
775                 s += A[k] * B[k, j]
776             result[j] = s
777         return result
778
779     # Case 4: matrix . matrix -> matrix
780     if A.ndim == 2 and B.ndim == 2:
781         rowsA, colsA = A.shape
782         rowsB, colsB = B.shape
783         if colsA != rowsB:
784             raise ValueError("Inner dimensions must match for matrix multiplication.")
785         result = np.zeros((rowsA, colsB), dtype=float)
786         for i in range(rowsA):
787             for j in range(colsB):
788                 s = 0.0
789                 for k in range(colsA):
790                     s += A[i, k] * B[k, j]
791                 result[i, j] = s
792         return result
793
794     raise ValueError("Unsupported input dimensions for matmul.")
795
796 def transpose(A):

```

```

797
798     A = np.asarray(A, dtype=float)
799
800     if A.ndim == 1:
801         # A 1D vector has no meaningful row/column transpose here;
802         # return a copy unchanged (consistent with numpy's own behavior for .T on 1D)
803         return A.copy()
804
805     if A.ndim == 2:
806         rows, cols = A.shape
807         result = np.zeros((cols, rows), dtype=float)
808         for i in range(rows):
809             for j in range(cols):
810                 result[j, i] = A[i, j]
811         return result
812
813     raise ValueError("transpose() only supports 1D or 2D arrays.")
814
815 n = X.shape[0]
816 C = C = matmul(transpose(X), X) / n # Compute covariance matrix
817
818 print("\n" + "=" * 70)
819 print("PART 3(a): Covariance matrix C = (1/n) X^T X")
820 print("=" * 70)
821 print("C =\n", C)
822 print("Shape of C:", C.shape)
823
824
825 # -----
826 # Q2(3b): Power Method for the largest eigenvalue and its vector
827 # -----
828 # Power Method idea:
829 #   Start with a non-zero vector v.
830 #   Repeatedly compute w = C v, normalize it, and repeat.
831 #   After enough iterations, v converges to the eigenvector corresponding
832 #   to the largest eigenvalue in magnitude.
833
834 # C is the covariance matrix from the previous cell.
835
836
837 def power_method(A, num_iterations=10000, tol=1e-12, verbose=False):
838     """Approximates the dominant eigenvalue and eigenvector of matrix A using the Power
839     Method.
840     Returns:
841         eigval      : approximated dominant eigenvalue
842         eigvec      : approximated dominant eigenvector (unit norm)
843         n_iterations : number of iterations actually performed
844     """
845
846     d = A.shape[0]
847     v = np.random.rand(d)
848     v = v / np.linalg.norm(v)
849
850     eigval_old = 0.0
851     for i in range(1, num_iterations + 1): # Loop until max iterations
852         w = matmul(A, v)
853         v_new = w / np.linalg.norm(w) # Normalize to get the new approx eigenvector
854
855         eigval_new = matmul(matmul(transpose(v_new), A), v_new) # Rayleigh quotient

```

```

855
856         if abs(eigval_new - eigval_old) < tol: # Checking convergence
857             v = v_new
858             eigval_old = eigval_new # Update estimate Eigenvalue
859             if verbose:
860                 print(f"Converged after {i} iterations.")
861             return eigval_new, v, i
862
863         v = v_new # Update eigenvector for next iteration
864         eigval_old = eigval_new
865
866     return eigval_old, v, num_iterations
867
868
869 lambda_1, v1, iterations_1 = power_method(C)
870
871 print("\n" + "=" * 70)
872 print("PART 3(b): Power Method - dominant eigenvalue/eigenvector")
873 print("=" * 70)
874 print("Lambda_1 (dominant eigenvalue):", lambda_1)
875 print("v_1 (dominant eigenvector):\n", v1)
876 print("Iterations taken:", iterations_1)
877
878
879
880 # -----
881 # 3(c). Deflation: find subsequent eigenvalues/eigenvectors
882 # -----
883
884
885 def outer(u, v):
886     """
887     Custom outer product of two 1D vectors u, v -> returns a 2D matrix.
888     result[i, j] = u[i] * v[j]
889     """
890     u = np.asarray(u, dtype=float)
891     v = np.asarray(v, dtype=float)
892     if u.ndim != 1 or v.ndim != 1:
893         raise ValueError("outer() expects two 1D vectors.")
894     result = np.zeros((u.shape[0], v.shape[0]), dtype=float)
895     for i in range(u.shape[0]):
896         for j in range(v.shape[0]):
897             result[i, j] = u[i] * v[j]
898     return result
899
900
901 def deflate_and_find_all(A, num_eigs, num_iterations=10000, tol=1e-12):
902     """Finds the top `num_eigs` eigenvalues and eigenvectors of a symmetric matrix A
903     using the Power Method with deflation:
904         C_1 = A
905         C_{k+1} = C_k - v_k v_k^T C_k
906     """
907     d = A.shape[0]
908     C_k = A.copy()
909     eigvals = []
910     eigvecs = []
911
912     for k in range(num_eigs): # Loop across all eigenvalues
913         lam, v, _ = power_method(C_k, num_iterations, tol)

```

```

913         eigvals.append(lam)
914         eigvecs.append(v)
915         # Deflate: remove the found component
916         C_k = C_k - matmul(outer(v, v), C_k)
917
918     return np.array(eigvals), np.array(eigvecs).T # eigvecs as columns
919
920
921 eigvals_pm, eigvecs_pm = deflate_and_find_all(C, num_eigs=6)
922
923 print("\n" + "=" * 70)
924 print("PART 3(c): All eigenvalues/eigenvectors via Power Method + Deflation")
925 print("=" * 70)
926 for k in range(6):
927     print(f"\nlambda_{k+1} = {eigvals_pm[k]:.6f}")
928     print(f"v_{k+1} = {eigvecs_pm[:, k]}")
929
930
931
932 # -----
933 # 3(d). Compare with NumPy's eigen-decomposition
934 # -----
935 # C is symmetric, so we use eig (returns eigenvalues in ascending order)
936 eigvals_np, eigvecs_np = np.linalg.eigh(C)
937
938 # Sort in descending order to match the power method ordering
939 idx_sort = np.argsort(eigvals_np)[::-1]
940 eigvals_np_sorted = eigvals_np[idx_sort]
941 eigvecs_np_sorted = eigvecs_np[:, idx_sort]
942
943 print("\n" + "=" * 70)
944 print("PART 3(d): Comparison with numpy.linalg.eigh")
945 print("=" * 70)
946 print(f"{'Index':<6}{'Power Method':<18}{'NumPy (eigh)':<18}{'Abs Diff':<12}")
947 for k in range(6):
948     diff = abs(eigvals_pm[k] - eigvals_np_sorted[k])
949     print(f"{k+1:<6}{eigvals_pm[k]:<18.6f}{eigvals_np_sorted[k]:<18.6f}{diff:<12.2e}")
950
951 print("\nEigenvectors (NumPy, columns, descending eigenvalue order):\n",
952       eigvecs_np_sorted)
953
954 print("\nNote: eigenvectors are unique only up to a sign (+/- v is still")
955 print("a valid eigenvector), so compare magnitudes / direction, not sign.")
956
957
958
959 # -----
960 # 3(e). Iterations required for accuracy of 1e-7 (power method vs numpy)
961 # -----
962 def power_method_track_iterations(A, true_eigval, accuracy=1e-7, max_iter=100000):
963     """
964     Runs the power method step-by-step and records the iteration at which
965     the eigenvalue estimate first gets within `accuracy` of the true
966     eigenvalue (obtained from numpy in part (d)).
967     """
968     d = A.shape[0]
969     v = np.random.rand(d)
970     v = v / np.linalg.norm(v)
971

```

```

972     for i in range(1, max_iter + 1): # Loop until max iterations
973         w = matmul(A, v)
974         v = w / np.linalg.norm(w)
975         eigval_est = matmul(matmul(transpose(v), A), v)
976
977         if abs(eigval_est - true_eigval) < accuracy: # Check if within desired accuracy
978             return i, eigval_est
979
980     return max_iter, eigval_est # did not converge within max_iter
981
982
983 print("\n" + "=" * 70)
984 print("PART 3(e): Iterations needed for accuracy of 1e-7")
985 print("=" * 70)
986 print("(Using the numpy eigenvalues from part (d) as the 'true' values)")
987
988 C_k = C.copy()
989 for k in range(6):
990     true_val = eigvals_np_sorted[k]
991     iters_needed, est_val = power_method_track_iterations(C_k, true_val)
992     print(f"\nEigenvalue {k+1}: true = {true_val:.6f}")
993     print(f"  Iterations to reach 1e-7 accuracy: {iters_needed}")
994     print(f"  Final estimate: {est_val:.10f}")
995
996     # Deflate using the eigenvector power method found for this eigenvalue,
997     # so the next iteration targets the next eigenvalue
998     lam_k, v_k, _ = power_method(C_k)
999     C_k = C_k - matmul(outer(v_k, v_k), C_k)
1000
1001
1002 """
1003
1004 Output:
1005 =====
1006 PART 1: Dataset generation
1007 =====
1008 Shape of X: (500, 6)
1009 First 5 rows of X:
1010 [[ 4.96714153e-01  9.26177548e-01  1.39935544e+00  7.78361076e-01
1011    3.77196095e+00 -1.57366716e-01]
1012  [-1.38264301e-01  1.90941664e+00  9.24633683e-01 -5.51185716e-01
1013    5.45172132e+00  2.02700512e+00]
1014  [ 6.47688538e-01 -1.39856757e+00  5.96303699e-02 -8.18198883e-01
1015   -2.90032565e+00  1.69602814e+00]
1016  [ 1.52302986e+00  5.62969237e-01 -6.46936778e-01 -3.37445741e-03
1017    4.73496742e+00 -6.40187863e-01]
1018  [-2.34153375e-01 -6.50642569e-01  6.98223314e-01 -1.70184623e-01
1019   -2.42023446e+00  1.03859256e+00]]
1020
1021 -> The first four columns are independent random features sampled from a standard normal
distribution.
1022 -> The last two columns are not independent because they are linear combinations of the
first four.
1023 -> X is a 500 x 6 matrix with maximum rank 4 according to theory.
1024
1025 =====
1026 PART 2: Rank of X
1027 =====
1028 Computed Rank from program of X: 4

```

```

1029 (Expected also 4, since f5 and f6 are linear combinations of f1..f4,
1030 so X has only 4 linearly independent columns.)
1031
1032 =====
1033 PART 3(a): Covariance matrix C = (1/n) X^T X
1034 =====
1035 C =
1036 [[ 9.60978978e-01 -7.22555486e-02 -5.64317918e-02  6.20376879e-02
1037    1.70519131e+00 -1.80507168e-01]
1038 [-7.22555486e-02  9.55578458e-01  7.84296908e-02 -1.96694541e-02
1039    2.72222428e+00  1.17768599e-01]
1040 [-5.64317918e-02  7.84296908e-02  1.03032538e+00 -1.82832955e-02
1041    1.22425489e-01  1.06689197e+00]
1042 [ 6.20376879e-02 -1.96694541e-02 -1.82832955e-02  9.67550808e-01
1043    6.50670136e-02 -1.95338491e+00]
1044 [ 1.70519131e+00  2.72222428e+00  1.22425489e-01  6.50670136e-02
1045    1.15770555e+01 -7.70853820e-03]
1046 [-1.80507168e-01  1.17768599e-01  1.06689197e+00 -1.95338491e+00
1047    -7.70853820e-03  4.97366179e+00]]
1048 Shape of C: (6, 6)
1049
1050 =====
1051 PART 3(b): Power Method - dominant eigenvalue/eigenvector
1052 =====
1053 Lambda_1 (dominant eigenvalue): 12.470201198104386
1054 v_1 (dominant eigenvector):
1055 [ 1.41318348e-01  2.26959199e-01  1.10861215e-02  5.94233543e-03
1056    9.63514294e-01 -7.98549331e-04]
1057 Iterations taken: 21
1058
1059 =====
1060 PART 3(c): All eigenvalues/eigenvectors via Power Method + Deflation
1061 =====
1062
1063 lambda_1 = 12.470201
1064 v_1 = [ 1.41318350e-01  2.26959198e-01  1.10861101e-02  5.94235589e-03
1065    9.63514294e-01 -7.98601637e-04]
1066
1067 lambda_2 = 5.981302
1068 v_2 = [-3.97193906e-02  2.65590072e-02  1.98654097e-01 -3.56511019e-01
1069    2.38240351e-04  9.11676135e-01]
1070
1071 lambda_3 = 1.054831
1072 v_3 = [-0.63152956  0.4163527  0.57452677  0.30921801 -0.01400101 -0.04390925]
1073
1074 lambda_4 = 0.958817
1075 v_4 = [ 0.54213247 -0.36273603  0.68093023  0.33253893 -0.00394313  0.01585237]
1076
1077 lambda_5 = 0.000000
1078 v_5 = [ 0.197255  0.33634867 -0.01343741  0.06660166  0.91833168  0.00325883]
1079
1080 lambda_6 = 0.000000
1081 v_6 = [ 0.11311376  0.27235551  0.03572256  0.07011813 -0.13195887  0.9430917 ]
1082
1083 =====
1084 PART 3(d): Comparison with numpy.linalg.eigh
1085 =====
1086 Index Power Method      NumPy (eigh)      Abs Diff
1087 1      12.470201          12.470201          1.44e-13

```

```

1088 2      5.981302      5.981302      1.39e-13
1089 3      1.054831      1.054831      4.13e-12
1090 4      0.958817      0.958817      4.14e-12
1091 5      0.000000      -0.000000      5.15e-15
1092 6      0.000000      -0.000000      9.10e-16
1093
1094 Eigenvectors (NumPy, columns, descending eigenvalue order):
1095 [[-1.41318356e-01  3.97193891e-02  6.31533121e-01 -5.42128328e-01
1096    4.02386784e-01 -3.51851051e-01]
1097 [-2.26959194e-01 -2.65590535e-02 -4.16355085e-01  3.62733292e-01
1098    6.03580176e-01 -5.27776576e-01]
1099 [-1.10860808e-02 -1.98654104e-01 -5.74522294e-01 -6.80934000e-01
1100    2.68730679e-01  3.07327983e-01]
1101 [-5.94240849e-03  3.56511015e-01 -3.09215835e-01 -3.32540952e-01
1102    -5.37461358e-01 -6.14655965e-01]
1103 [-9.63514294e-01 -2.38382275e-04  1.40009874e-02  3.94322148e-03
1104    -2.01193392e-01  1.75925525e-01]
1105 [ 7.98736164e-04 -9.11676134e-01  4.39093752e-02 -1.58520962e-02
1106    -2.68730679e-01 -3.07327983e-01]]
1107
1108 Note: eigenvectors are unique only up to a sign (+/- v is still
1109 a valid eigenvector), so compare magnitudes / direction, not sign.
1110
1111 =====
1112 PART 3(e): Iterations needed for accuracy of 1e-7
1113 =====
1114 (Using the numpy eigenvalues from part (d) as the 'true' values)
1115
1116 Eigenvalue 1: true = 12.470201
1117     Iterations to reach 1e-7 accuracy: 13
1118     Final estimate: 12.4702011344
1119
1120 Eigenvalue 2: true = 5.981302
1121     Iterations to reach 1e-7 accuracy: 7
1122     Final estimate: 5.9813017676
1123
1124 Eigenvalue 3: true = 1.054831
1125     Iterations to reach 1e-7 accuracy: 88
1126     Final estimate: 1.0548309013
1127
1128 Eigenvalue 4: true = 0.958817
1129     Iterations to reach 1e-7 accuracy: 1
1130     Final estimate: 0.9588168977
1131
1132 Eigenvalue 5: true = -0.000000
1133     Iterations to reach 1e-7 accuracy: 1
1134     Final estimate: 0.0000000000
1135
1136 Eigenvalue 6: true = -0.000000
1137     Iterations to reach 1e-7 accuracy: 1
1138     Final estimate: 0.0000000000
1139
1140 ""

```